

INITIATION AU LOGICIEL MATLAB

MOHAMED SAAD BOUH ELEMINE VALL

Institut Supérieur de Comptabilité et d'Administration des Entreprises (ISCAE)
Licence RIT-DI-IG- S_3

Année universitaire 2023-2024



1 Introduction



1 Introduction

2 Concepts de base de MATLAB

- Variables, opérations et scripts
- Graphisme



Introduction

Utilité

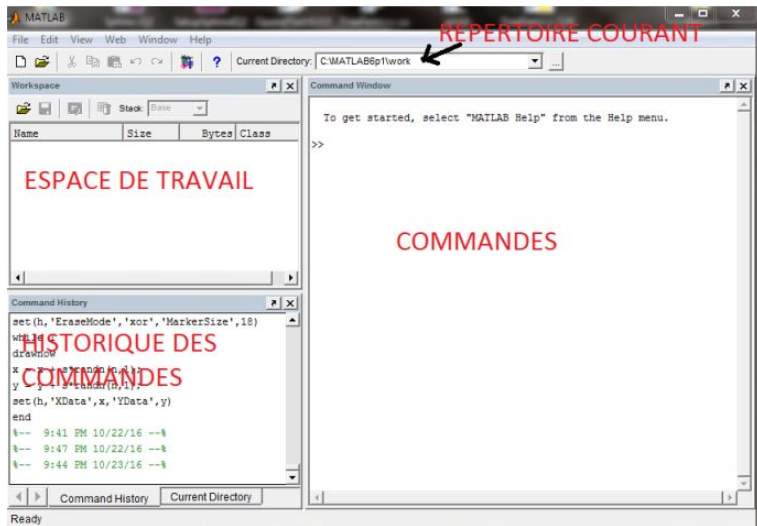
Simulation numérique des phénomènes de la physique, chimie, biologie, et sciences appliquées en général, afin de prédire, comprendre, optimiser, voire contrôler leurs comportements.

Types de logiciels

- Logiciels de calcul formel : donnent les solutions sous forme symbolique (ex. $f'(x) = 3x$) ou représentation graphique.
(Mathematica, Maple, Matlab, Maxima, ..., etc)
- Logiciels de simulation : fournit des solutions numériques ou représentations graphiques.
(Matlab, Scilab, Octave, Python, R, ..., etc.)



Environnement Matlab



Introduction

Avantages d'utilisation de Matlab

- ❶ Facilité d'utilisation : MATLAB offre une syntaxe simple et facile à apprendre, ce qui le rend accessible aux débutants en programmation et aux utilisateurs non techniques.
- ❷ Large gamme de fonctionnalités : MATLAB propose une vaste collection de bibliothèques et de boîtes à outils pour diverses applications scientifiques telles que le traitement du signal, l'analyse de données, la modélisation mathématique, et bien plus encore.
- ❸ Visualisation avancée : Il permet de créer des représentations visuelles efficaces des données, ce qui facilite la compréhension des modèles et des résultats.
- ❹ Intégration facile : MATLAB peut être facilement intégré à d'autres langages de programmation et à des logiciels tiers, ce qui facilite l'interopérabilité avec d'autres outils et applications.

Introduction

- 1 Grande communauté et support : MATLAB bénéficie d'une grande communauté d'utilisateurs actifs, ce qui facilite l'accès à des ressources d'apprentissage, des forums de discussion et des solutions aux problèmes courants.
- 2 Performances élevées : MATLAB est optimisé pour les calculs scientifiques complexes, ce qui lui permet de manipuler des ensembles de données volumineux et d'exécuter des opérations mathématiques intensives de manière efficace.
- 3 Développement rapide de prototypes : MATLAB facilite le développement rapide de prototypes et d'applications en permettant aux utilisateurs de tester rapidement des idées et des algorithmes avant une implémentation plus complexe.



Types des variables Matlab

- ❶ Pas besoin d'initialiser les variables.
- ❷ Matlab supporte différents types de variables, exemples :
 - réels double précision (défaut) : ex. `>> 4.56`
 - chaînes de caractères (16-bits) : ex. `>> 'a'`
- ❸ La plupart des variables sont des vecteurs, des matrices de réels double-précision ou des chaînes de caractères.
- ❹ Autres types supportés : les nombres complexes et le calcul symbolique.
- ❺ Déclaration de variables :
 - `>> var1=3.14 ;`
 - `>> chaine='ISCAE' ;`
- ❻ Pour les noms de variables : une lettre comme premier caractère, suivie par une combinaison de lettres et/ou chiffres (Attention : $Var1 \neq var1$).
- ❼ Faut pas utiliser les noms de constantes ou fonctions prédéfinies (i, j, pi, sin, cos, ..., etc)

Concepts de base de MATLAB

- ❶ Vecteur ligne : une virgule ou un espace sépare les composantes entre crochets :
 - ❶ `>> ligne=[1 2 -5 3.2];`
 - ❷ `>> ligne = [3, 64, -2];`
- ❷ Vecteur colonne : point virgule entre les composantes
`>> colonne = [-6; 3; 9; -10];`
- ❸ Pour distinguer entre vecteurs lignes et vecteurs colonnes :
 - voir le workspace (espace de travail)
 - afficher la variable dans la fenêtre des commandes
 - utiliser la fonction `size`
- ❹ La fonction `length` renvoie la longueur d'un vecteur (ligne ou colonne).



Concepts de base de MATLAB

1 Création des vecteurs par concaténation :

```
>> a = [1 2];
```

```
>> b = [3 4];
```

```
>> c = [5; 6];
```

```
>> d = [a; b];
```

```
>> e = [d c];
```

```
>> f = [[e e]; [a b a]];
```

```
>> str = ['bonjour, je suis' 'Mohamed'];
```

: les chaînes sont des vecteurs de caractères.



Concepts de base de MATLAB

Sauvegarde/nettoyage/chargement

- 1 Utiliser **save** pour sauvegarder les variables dans un fichier :
`>> save monFichier a b` : sauvegarde les variables **a**, **b** dans le fichier **monFichier.dat** dans le repertoire courant.
- 2 Utiliser **clear** pour supprimer les variables :
`>> clear a b` : supprime a, b de l'espace de travail.
- 3 Utiliser **load** pour charger les variables sauvegardées dans un fichier :
`>> load monFichier`



Concepts de base de MATLAB

Opérations de base sur les scalaires

- ❶ Opérations arithmétiques (+, -, /, *) :

```
>> 6/4
```

```
>> (1+i)*(2-i)
```

```
>> 1/0
```

```
>> 0/0
```

- ❷ la multiplication n'est pas implicite :

```
>> 2(1+5*i) produit une erreur !
```

- ❸ Utilisez `clc` pour effacer la fenêtre de commandes.

- ❹ Matlab respecte les règles de calcul matriciel :

La commande `>> c = ligne + colonne` suivante donne une erreur. Donc, faut utiliser la transposée pour que les dimensions soient compatibles :

```
>> c = ligne' + colonne
```

```
>> c = ligne + colonne'
```

- ❺ Nous pouvons additionner ou multiplier les composantes d'un vecteur :

```
>> s = sum(ligne)
```

```
>> p = prod(ligne)
```

Concepts de base de MATLAB

Fonctions prédéfinies en Matlab

- 1 Matlab possède une vaste librairie de fonctions prédéfinies (`sin`, `cos`, `exp`, ...)
- 2 Utiliser `help` pour voir l'aide :
`>> help sin` : liste les fonctions liées à la fin de l'aide.
`>> doc sin` : donne des exemples.
- 3 Pour appeler une fonction, on fournit ses paramètres entre parenthèses :
`>> sqrt(2)`
`>> log(2)`
`>> exp(1+i)`
- 4 Les fonctions sur les scalaires sont définies pour les vecteurs :
`>> t = [1 2 3]`
`>> f = exp(t)`
- 5 en plus du mode standard, les opérateurs (`+`, `-`, `/`, `*`) fonctionnent en mode élément par élément (`./`, `.-`, `./`, `.*`) :
`>> a=[1 2 3]; b=[4;2;1];`
`>> a.*b, a./b` : erreurs, `>> a.*b'`, `a./b'` : correctes

Concepts de base de MATLAB

Initialisation automatique et matrices particulières

- 1 Utiliser les fonctions **ones**, **zeros** et **rand** pour initialiser les matrices avec des 1, des zeros ou des nombres aléatoires respectivement.
`>> v = ones(1,10)` : vecteur ligne de 10 composantes égales à 1
`>> t = zeros(12,1)` : vecteur colonne de 12 composantes égales à zéro
`>> M = ones(3,4)` : matrice 3×4 de coefficients 1
- 2 on peut initialiser un vecteur avec **linspace**
`>> a = linspace(0,10,5)` : première composante zéro, dernière composante 10 (comprise), 5 composantes.
- 3 on peut aussi utiliser (:) `>> b = 0 :2 :10`
- 4 Pour initialiser avec des valeurs logarithmiquement espacées, utiliser **logspace** (voir help).
- 5 les indices des vecteurs commencent à 1, non 0
`>> a = [5 2 3], a(1) = 5`

Concepts de base de MATLAB

- ❶ Le paramètre des indices peut être un vecteur

```
>> x = [3 6 9 10]
```

```
>> a = x(2 :3)
```

```
>> b = x(1 :end-1)
```

- ❷ Extraction de sous-matrices >> `a = rand(5)` : matrice 5×5

```
>> b = a(1 :3,1 :2) : sous matrice
```

```
>> c = a([1 5 3],[1 4]) : lignes et colonnes spécifiques.
```

- ❸ Utiliser (:) pour sélectionner des lignes ou colonnes d'une matrice

```
>> c = [12 5; -2 13]
```

```
>> d = c(1, :) : ligne 1 de c
```

```
>> e = c(:,2) : colonne 2 de c
```

```
>> c(2, :)= [3 4] : remplace ligne 2 de c.
```



Concepts de base de MATLAB

- 1 Matlab comporte des fonctions pour aider à la recherche d'éléments d'un vecteur ou matrice

```
>> vec = [5 3 1 9 7]
```

```
>> [minVal, minInd] = min(vec) : le minimum et son indice (marche pour le max).
```

- 2 Pour trouver les indices de valeurs spécifiques

```
>> ind = find(vec == 9)
```

```
>> ind = find(vec > 2 & vec < 6)
```



Concepts de base de MATLAB

Scripts en Matlab

- ❶ Les scripts sont :
 - ❶ un ensemble de commandes exécutées de manière séquentielle
 - ❷ les scripts sont écrits dans l'éditeur de Matlab
- ❷ Ils sont sauvegardés dans un fichier Matlab (extension .m)
- ❸ Pour créer un script :
 - ❶ `>> edit premScript.m`, à partir de la fenêtre des commandes
 - ❷ clic sur le-nouveau (ou sur l'icone de création)
- ❹ Commentaires :
 - ❶ le reste d'une ligne après % est considéré comme un commentaire
 - ❷ les commentaires permettent d'éviter la perte de temps plus tard
- ❺ Les scripts sont statics : pas d'entrée ni de sortie explicite
- ❻ Toutes les variables créées ou modifiées par le script existent dans l'espace de travail (workspace)
- ❼ Utiliser `disp` pour afficher à l'écran : `disp('calcul scientifique')`

Concepts de base de MATLAB

- 1 Exemple :
`x=linspace(0,4*pi,10);`
`y = sin(x);`
`plot(y)`, trace le vecteur y en fonction des indices
- 2 Pour tracer y en fonction de x : `plot(x,y)`
- 3 Pour un tracé plus régulier (lisse), on utilise plus de points :
`x=linspace(0,4*pi,1000);`
`plot(x,sin(x))`
- 4 Pour représenter une fonction de 2 variables $f(x, y)$, on doit :
 - définir un maillage du plan (x, y)
 - évaluer la fonction f aux noeuds du maillage
 - utiliser `mesh`, `surf`, `contour`, `meshc` pour la représentation graphique
- 5 Exemple pour $f(x, y) = ye^{-(x^2+y^2)}$ dans le domaine $D = [-2, 2]^2$:
`x = -2 :2 ; y = -1 :2`, quadrillage des axes
`[X,Y] = meshgrid(x,y)`, maillage
`Z = Y .* exp(-(X.^2 + Y.^2))`, évaluation de la fonction aux noeuds du maillage
`mesh(X,Y,Z)`, tracé de la fonction

Concepts de base de MATLAB

- ❶ Toute matrice peut être représentée par une image (`imagesc(mat)`)
- ❷ Les fonctions définies par l'utilisateur sont écrites dans l'éditeur de MATLAB, comme les scripts (extension `.m`)
- ❸ Leurs variables sont locales : leurs valeurs sont accessibles seulement à la fonction
- ❹ Exemple :

```
function y=sind(x)  
% sind(x) est le sinus en degrés de x  
y=sin(x*pi/180);  
end
```
- ❺ Le nom du fichier de sauvegarde est le même que celui de la fonction (`sind.m`)
- ❻ L'exécution de la fonction se fait dans la fenêtre des commandes :

```
>> sind(10)
```
- ❼ Syntaxe générale de création d'une fonction est :

```
function [variables de sortie] = nomfonction(paramètres d'entrée)  
Instructions  
end
```

Concepts de base de MATLAB

- 1 Les fonctions communiquent avec l'espace de travail (workspace) seulement via les paramètres d'entrée et variables sorties
- 2 Séparer les entrées et sorties par des virgules, s'ils sont plusieurs les premières lignes doivent être des commentaires (affichés par help)
`>> help sind`
- 3 Le corps de la fonction doit comporter des instructions qui affectent des valeurs aux variables de sortie
- 4 Une fonction définie pour des scalaires peut opérer sur des tableaux :
`>> longueurs = 100*sind([30 60 90 ; 120 150 180])`
- 5 Les fonctions Matlab sont généralement surchargées : peuvent accepter un nombre variable de paramètres (entrées) peuvent retourner un nombre variable de sorties
- 6 Les fonctions définies par l'utilisateur peuvent être surchargées via `varargin`, `nargin`, `varargout`, `nargout`

Concepts de base de MATLAB

- ① Opérateurs standards
== (égale), != (différent), > (supérieur), < (inférieur), >= (supérieur ou égal), <= (inférieur ou égal)
- ② Opérateurs logiques & (et), | (ou), ~ (non), all (toutes vraies), any (toutes fausses), && (et pour scalaires), || (ou pour scalaires)
- ③ Booléens 0 (faux), non nul (vrai)
- ④ Structure if :
if condition
instructions
end
- ⑤ Structure else :
if condition
instructions1
else
instructions2
end

Concepts de base de MATLAB

- ❶ Structure `elseif` :
`if condition1`
`instructions1`
`elseif condition2`
`instructions2`
`else`
`instructions3 end`
- ❷ Utiliser `for` pour un nombre connu d'itérations syntaxe :
- ❸ Le block des instructions est tout ce qui est compris entre `for` et `end`.
- ❹ Avec la boucle `while`, on n'a pas besoin de connaître le nombre d'itérations syntaxe :
`while condition`
`instructions`
`end`
- ❺ Les instructions seront exécutées tant que la condition est vraie.

Concepts de base de MATLAB

❶ Éviter, dans la mesure du possible, l'utilisation de boucles

❷ La vectorisation du code est plus efficace dans Matlab :

```
>> a = rand(1,100);  
>> b = zeros(1,100);  
>> for n=1 :100  
if n == 1  
b(n) = a(n);  
else  
b(n) = a(n-1) + a(n);  
end  
end
```

❸ Code plus efficace et plus propre :

```
>> a = rand(1,100);  
>> b = [0 a(1 :end-1)] + a;
```

Concepts de base de MATLAB

Polynômes en Matlab

- 1 Matlab représente un polynôme par le vecteur de ces coefficients :
 $ax^3 + bx^2 + cx + d$, $p(1) = a$, $p(2) = b$, $p(3) = c$, $p(4) = d$
 $P = [1 \ 0 \ -2]$ représente le polynôme $x^2 - 2$
 $P = [2 \ 0 \ 0 \ 0]$ représente le polynôme $2x^3$
- 2 P est un vecteur de $N + 1$ composantes qui décrit un polynôme d'ordre N
- 3 utiliser **roots** pour calculer les racines de P :
`>> r = roots(P)`, où $\dim(r)=N$
- 4 Pour déterminer un polynôme à partir de ses racines :
`>> P = poly(r)`
- 5 Pour évaluer un polynôme en un point :
`>> y0 = polyval(P,x0)`, où x_0 , y_0 scalaires
- 6 Pour évaluer un polynôme en plusieurs points :
`>> y = polyval(P,x)`, où x , y vecteurs de mêmes dimensions

Concepts de base de MATLAB

- 1 Matlab permet d'approximer un ensemble de données par des polynômes étant données les données $X = [-1 \ 0 \ 2]$ et $Y = [0 \ -1 \ 3]$
`>> p2 = polyfit(X,Y,2)`, donne le meilleur polynôme d'ordre 2 qui approxime les points $(-1,0), (0,-1), (2,3)$
- 2 Représentation graphique :
`>> plot(X,Y,'o','MarkerSize',10)`
`>> hold on`
`>> x = -3 :0.01 :3`
`>> plot(x,polyval(p2,x),'r-')`
- 3 Plusieurs problèmes pratiques nécessitent la résolution de $f(x) = 0$, où f est une fonction donnée
- 4 Utiliser `fzero` pour le calcul des racines d'une fonction arbitraire
- 5 `fzero` nécessite de passer la fonction en paramètre : écrire la fonction dans un fichier : `function y = mafonc(x)`
`y = cos(exp(x))+x2-1`
`>> x = fzero('mafonc',1)`, où 1 est une première approximation de la racine
`>> x = fzero(@mafonc,1)`

Concepts de base de MATLAB

Optimisation en Matlab

- 1 Utiliser `fminbnd` pour minimiser une fonction sur un intervalle borné :
exemple `x=fminbnd('mafunc',-1,2)`
- 2 `fminsearch`, donne le minimum local sans contrainte d'intervalle :
Exemple `x = fminsearch('mafunc',.5)`, donne le minimum local de *mafunc* en partant de $x = 0.5$.



Concepts de base de MATLAB

Différentiation en Matlab

- 1 Matlab peut calculer numériquement la dérivée d'une fonction :

```
>> x = 0 :0.01 :2*pi ;  
>> y = sin(x) ;  
>> dydx =diff(y)./diff(x) ;  
>> x1 = x(1 :end-1) ;  
>> plot(x,y,x1,dydx)
```

- 2 La différentiation s'applique aux matrices :

```
>> mat = [1 3 5 ;4 6 8] ;  
>> dm = diff(mat,1,2) ; donne dm=[2 2 ;2 2] première différence par  
rapport à la seconde dimension
```

- 3 L'opposé de `diff` est la somme cumulative `cumsum`



Concepts de base de MATLAB

Intégration numérique en Matlab

1 Matlab contient les méthodes classiques d'intégration numérique

2 Quadrature de Simpson (une fonction en entrée) :

`>> q = quad('mafunc',0,10)` ; intégrale de la fonction 'mafunc' entre 0 et 10

`>> q2 = quad(@(x) x*sin(x),0,pi)` ; est $\int_0^{\pi} x \sin(x) dx$

3 Formule du trapèze (un vecteur en entrée) : `>> x = 0 :0.01 :pi` ;

`>> z = trapz(x,sin(x))` ; est $\int_0^{\pi} x \sin(x) dx$

`>> z2 = trapz(x,sqrt(exp(x))./x)` ; est $\int_0^{\pi} \frac{e^{x/2}}{x} dx$



Concepts de base de MATLAB

Équations différentielles ordinaires sous Matlab

- 1 Matlab implémente les solveurs classiques des équations différentielles, exemple :
 - `ode23`, solveur d'ordre inférieur : utilisé si l'intervalle de définition de la solution est petit ou la précision moins importante que la vitesse.
 - `ode45`, solveur d'ordre supérieur : le plus utilisé, grande précision et vitesse raisonnable.
- 2 Utilisation des solveurs EDO :
 - `[t,y] = ode45('monEDO',[0,10],[1;0])` ;
 - entrées : `'monODE'` définit l'EDO, `[0, 10]` intervalle des temps, `[1;0]` conditions initiales
 - sorties : `t` vecteur temps et `y` vecteur solution



Concepts de base de MATLAB

Exemple :

- 1 utiliser `ode45` pour résoudre l'équation suivante et donner la représentation graphique de la solution :

$$\begin{cases} \frac{dy(t)}{dt} = -ty/10, & t \in [0, 1] \\ y(0) = 10. \end{cases}$$

- 2 Écrire le fichier de définition de l'équation :

```
function dydt = monODE(t,y)
dydt = -t*y/10;
end
```

- 3 intégrer l'EDO :

```
>> [t,y] = ode45('monODE',[0,1],10)
```

- 4 représentation graphique :

```
>> plot(t,y); xlabel('temps'); ylabel('y(t)');
```

Concepts de base de MATLAB

Calcul de la limite en Matlab

- 1 Matlab utilise la commande `limit` pour calculer la limite d'une fonction, si la variable tend vers 0 :

```
>> syms x  
>> limit((x3 + 5)/(x4 + 7))
```

- 2 si $x \rightarrow 3$:

```
>> limit(x2 + 5, 3)
```

- 3 si $x \rightarrow 1$ >> `limit((x-3)/(x-1),1)`

- 4 limite à gauche et limite à droite :

```
>> f = (x-3)/abs(x-3)  
>> ezplot(f,[-1,5]) : représentation graphique  
>> l = limit(f,x,3,'left')  
>> r = limit(f,x,3,'right')
```



Concepts de base de MATLAB

Dérivation symbolique en Matlab

- ❶ la commande **diff** permet de calculer la dérivée symbolique d'une fonction :

```
>> syms t
```

```
>> f = 3*t^2+2*t^(-2)
```

```
>> diff(f) diff(f,n) pour les dérivées d'ordre supérieur :
```

```
>> f = x*exp(-3*x)
```

```
>> diff(f,2)
```

```
>> pretty(diff(f,2)), pour un format plus lisible
```



MERCI POUR VOTRE ATTENTION

